

Шпаргалка

Синтаксис TypeScript, React и Python и правило, когда какой приём уместен. Примеры рассчитаны на TypeScript 5, React 18 и Python 3.10+.

Когда что применять

Одна и та же задача выглядит по-разному в типе, в интерфейсе и в скрипте. Ниже — выбор приёма, а не полный синтаксис.

Несколько взаимоисключающих состояний

Когда: экран или операция бывает «ожидание», «загрузка», «успех» или «ошибка». Отдельные флаги `isLoading` и `error` легко оказываются истинными одновременно.

TYPESCRIPT

```
type State<T> =  
  | { status: "idle" }  
  | { status: "loading" }  
  | { status: "ok"; data: T }  
  | { status: "error"; message: string };
```

REACT

```
const [state, setState] =  
  useState<State<User>>({ status: "idle" });  
  
switch (state.status) {  
  case "ok":  
    return <p>{state.data.name}</p>;  
}
```

PYTHON

```
class Status(Enum):  
    IDLE = "idle"  
    LOADING = "loading"  
    ERROR = "error"  
  
match status:  
    case Status.ERROR:  
        ...
```

Входным данным нельзя верить

Когда: значение пришло из JSON, формы, файла или аргумента командной строки. Внутри своей функции после проверки тип уже можно считать известным.

TYPESCRIPT

```
function readName(input: unknown): string {
  if (typeof input !== "string" || input === "") {
    throw new Error("Имя обязательно");
  }
  return input;
}
```

REACT

```
{user ? (
  <Profile user={user} />
) : (
  <p>Нет данных</p>
)}
```

PYTHON

```
try:
    age = int(raw_age)
except ValueError:
    age = None
```

Одна логика для разных типов

Когда: функция не знает конкретный тип заранее, но должна вернуть тот же тип, что получила. Если тип всегда один, обобщение не нужно.

TYPESCRIPT

```
function first<T>(items: T[]): T | undefined {  
  return items[0];  
}
```

REACT

```
function useToggle(initial = false) {  
  const [on, setOn] = useState(initial);  
  const toggle = () => setOn((v) => !v);  
  return { on, toggle };  
}
```

PYTHON

```
from typing import TypeVar  
  
T = TypeVar("T")  
  
def first(items: list[T]) → T | None:  
    return items[0] if items else None
```

Два компонента смотрят на одни данные

Когда: список и счётчик выбранных живут рядом — состояние поднимают к общему родителю. Тема или текущий пользователь нужны десятку далёких компонентов — тогда контекст. Поле поиска, которое меняется на каждую букву, в контекст класть не стоит: перерисуются все подписчики.

БЛИЗКИЕ СОСЕДИ

```
function Page() {
  const [id, setId] = useState<string | null>(null);
  return (
    <
      <List onSelect={setId} />
      <Details id={id} />
    </>
  );
}
```

ДАЛЁКИЕ КОМПОНЕНТЫ

```
const UserContext = createContext<User | null>(null);

function useUser() {
  const user = useContext(UserContext);
  if (!user) throw new Error("Нет провайдера");
  return user;
}
```

НЕ REACT

```
# В Python и в чистом TypeScript
# общие данные — аргумент функции
# или поле объекта, а не глобальная
# переменная.
```

Ресурс нужно закрыть

Когда: открыт файл, сокет, таймер или запрос. Заккрытие должно случиться и при ошибке, и когда компонент исчез.

REACT

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id);  
}, []);
```

PYTHON

```
with open(path, encoding="utf-8") as file:  
    return file.read()
```

TYPESCRIPT

```
const timer = setTimeout(finish, 300);  
try {  
  await work();  
} finally {  
  clearTimeout(timer);  
}
```

Преобразовать коллекцию

Когда: из списка нужно получить другой список или словарь одной понятной операцией. Если шагов несколько и есть ранний выход, обычный цикл читается легче.

TYPESCRIPT

```
const names = users
  .filter((user) => user.active)
  .map((user) => user.name);
```

REACT

```
const names = useMemo(
  () => users.filter((u) => u.active).map((u) => u.name),
  [users]
);
```

PYTHON

```
names = [user.name for user in users if user.active]
by_id = {user.id: user for user in users}
```

TypeScript

Типы описывают контракт на границе функции и модуля. Локальной переменной аннотация часто не нужна: компилятор выводит её сам.

Базовые типы

тип

Когда: аннотируйте параметры и возвращаемое значение публичной функции. Массив пишется как `string[]`, пара фиксированной длины — как кортеж.

```
let title: string = "Шпаггалка";
let year: number = 2026;
let published: boolean = true;
let tags: string[] = ["react", "python"];
let point: [number, number] = [10, 20];
```

Функции и необязательный аргумент

функция

Когда: ? означает «можно не передавать». Возвращаемый тип у экспортируемой функции лучше написать явно, чтобы смена реализации не молча поменяла контракт.

```
function greet(name: string, title?: string): string {  
  return title ? `${title} ${name}` : name;  
}  
  
const area = (width: number, height: number): number ⇒ width * height;
```

interface и type

контракт

Когда: interface — форма объекта, которую можно расширять через extends. type — объединение, пересечение, примитив и всё, что не является одним объектом.

```
interface User {  
  id: string;  
  name: string;  
  email?: string;  
}  
  
type Id = string | number;  
type Admin = User & { role: "admin" };
```

email? — поле можно не указывать. Это не то же самое, что email: string | null: null надо передать явно.

Сужение union

union

Когда: значение бывает одним из нескольких литералов. После switch или проверки typeof TypeScript оставляет только подходящую ветку.

```
type Status = "idle" | "loading" | "error";

function label(status: Status): string {
  switch (status) {
    case "idle":
      return "Ожидание";
    case "loading":
      return "Загрузка";
    case "error":
      return "Ошибка";
  }
}
```

unknown вместо any

граница

Когда: данные пришли снаружи. unknown заставляет проверить тип перед использованием. any выключает проверку — его оставляют только как временную метку в чужом коде, который ещё не описан.

```
function isUser(value: unknown): value is User {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value &&
    "name" in value
  );
}
```

Generics

обобщение

Когда: алгоритм одинаковый, а тип элемента разный, и результат должен сохранить этот тип. Для одной конкретной структуры generic только шумит.

```
function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}

const id = first([ { id: "a" }, { id: "b" } ])?.id;
```

Служебные типы

Partial Pick Omit

Когда: Partial — тело PATCH, где любое поле необязательно. Pick и Omit — урезанный вид для ответа API. Record — словарь с известным типом ключа и значения.

```
type UserUpdate = Partial<User>;
type PublicUser = Pick<User, "id" | "name">;
type WithoutEmail = Omit<User, "email">;
type UsersById = Record<string, User>;
```

as const и readonly

литерал

Когда: список строк должен стать объединением литералов, а не просто string[]. readonly запрещает переписывание поля на этапе компиляции, в рантайме объект остаётся обычным.

```
const roles = ["admin", "editor", "user"] as const;
type Role = (typeof roles)[number];

interface Account {
  readonly id: string;
  role: Role;
}
```

React

Компонент — функция от props к разметке. Состояние хранит то, что меняется из-за действий пользователя. Всё, что можно вычислить из уже имеющихся данных, вычисляют во время рендера.

Компонент, props и children

JSX

Когда: данные приходят от родителя — это props. children — слот для вложенной разметки. Так собирают карточку или лейаут, не плодя флаги showFooter.

```
import type { ReactNode } from "react";

type CardProps = {
  title: string;
  children: ReactNode;
};

function Card({ title, children }: CardProps) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
    </section>
  );
}
```

useState

состояние

Когда: значение живёт внутри компонента и меняется со временем: открыта панель, выбранный id, текст поля. Если новое значение зависит от старого, передавайте функцию — так не потеряется промежуточное обновление.

```
const [count, setCount] = useState(0);
const [user, setUser] = useState<User | null>(null);

setCount((prev) => prev + 1);
```

Начальное значение `useState(getItems())` вызовет `getItems` на каждый рендер. Для дорогой инициализации передают функцию: `useState(() => getItems())`.

Условие и список

key

Когда: ветка рендера зависит от данных — тернарный оператор или `&&`. В списке `key` — стабильный id записи. Индекс подходит только списку, который не сортируют, не фильтруют и не удаляют из середины.

```
{user ? <Profile user={user} /> : <Login />}

<ul>
  {items.map((item) => (
    <li key={item.id}>{item.name}</li>
  ))}
</ul>
```

Контролируемый ввод

форма

Когда: значение нужно на каждый ввод: фильтр, валидация, счётчик символов. Если прочитать поле достаточно в момент отправки, хватит неконтролируемого поля и FormData.

```
const [query, setQuery] = useState("");



function onSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault();
  const data = new FormData(event.currentTarget);
}
```

useEffect

эффект

Когда: компонент синхронизируется с внешним миром: сеть, подписка, таймер, документ. Функция, которую возвращает эффект, — очистка. Зависимость [id] значит «запустить снова, когда сменился id». Полное имя из имени и фамилии считают в теле компонента, без эффекта.

```
useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/users/${id}`, { signal: controller.signal })
    .then((response) => response.json())
    .then(setUser);

  return () => controller.abort();
}, [id]);
```

useMemo и useCallback

memo

Когда: расчёт заметно дорогой или ссылку на функцию/массив нельзя менять на каждый рендер, потому что её ждёт React.memo или зависимость эффекта. Оборачивать каждый обработчик заранее не нужно.

```
const visible = useMemo(
  () => items.filter((item) => item.name.includes(query)),
  [items, query]
);

const onSelect = useCallback((nextId: string) => {
  setSelectedId(nextId);
}, []);
```

Для короткого списка фильтр прямо в теле компонента проще. useMemo оставляют, когда расчёт дорогой или ссылку нельзя менять на каждый рендер.

Свой хук

хук

Когда: одна и та же связка состояния и обработчиков нужна в нескольких компонентах. Если связка используется один раз, оставьте её в компоненте — отдельный хук не сделает код короче.

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => {
    setOn((value) => !value);
  }, []);
  return { on, toggle };
}
```

Имя хука начинается с use. Вызывают его только на верхнем уровне компонента, не внутри условия и не внутри цикла.

Python

Аннотации помогают редактору и читателю, но в рантайме сами по себе ничего не проверяют. Проверку недоверенного ввода делают явно: `int()`, разбор JSON, своя функция.

Переменные и аннотации

тип

Когда: аннотируйте аргументы и результат функции, которую вызывают из другого модуля. В коротком скрипте локальные имена и так читаются.

```
name: str = "Борис"
age: int = 28
scores: list[int] = [10, 20, 30]
user: dict[str, str] = {"name": name}
```

Функция и значение по умолчанию

def

Когда: параметр можно опустить — дайте ему значение по умолчанию. Список и словарь в значении по умолчанию нельзя: объект создаётся один раз и делится между вызовами. Для «пустой коллекции» ставьте `None` и создавайте список внутри.

```
def greet(name: str, title: str | None = None) → str:
    if title:
        return f"{title} {name}"
    return name

def add_tag(tags: list[str] | None = None) → list[str]:
    if tags is None:
        tags = []
    tags.append("new")
    return tags
```

list, tuple, dict, set

коллекция

Когда: list — порядок и возможные повторы. tuple — короткая неизменяемая запись, её можно положить в множество. dict — поиск по ключу. set — уникальные элементы и проверка «есть ли».

```
names = ["Анна", "Борис", "Анна"]
unique = set(names)
person = ("Борис", 28)
user = {"name": "Борис"}

email = user.get("email", "-") # ключа может не быть
name = user["name"]           # ключ обязан быть
```

Генераторы коллекций

comprehension

Когда: одно преобразование или фильтр. Цикл for — если внутри несколько шагов, запись в несколько мест или выход по break.

```
names = [user.name for user in users if user.active]
by_id = {user.id: user for user in users}
adult_ids = {user.id for user in users if user.age ≥ 18}
```

enumerate и zip

обход

Когда: enumerate — нужен номер элемента. zip — две последовательности идут парой. strict=True поднимет ошибку, если длины различаются, вместо тихого обрезания.

```
for index, name in enumerate(names, start=1):
    print(index, name)

for name, score in zip(names, scores, strict=True):
    print(f"{name}: {score}")
```

dataclass

класс

Когда: объект в первую очередь хранит поля. Обычный класс — когда есть инвариант, который нельзя нарушить присваиванием, и методы, которые этот инвариант держат. `frozen=True` запрещает менять поля после создания.

```
from dataclasses import dataclass

@dataclass
class User:
    id: str
    name: str
    email: str | None = None
```

match

состояние

Когда: у разных состояний разный набор полей, как у union в TypeScript. Enum достаточно, если состояний несколько, а дополнительных данных нет.

```
from enum import Enum

class Status(Enum):
    IDLE = "idle"
    LOADING = "loading"
    ERROR = "error"

def label(status: Status) → str:
    match status:
        case Status.IDLE:
            return "Ожидание"
        case Status.LOADING:
            return "Загрузка"
        case Status.ERROR:
            return "Ошибка"
```

Файлы и with

ресурс

Когда: ресурс обязан закрыться, даже если внутри случилась ошибка. Path.read_text удобен для небольшого файла целиком. Построчное чтение большого файла оставляют в with.

```
from pathlib import Path

text = Path("notes.txt").read_text(encoding="utf-8")

with open("notes.txt", encoding="utf-8") as file:
    for line in file:
        print(line.rstrip())
```

Исключения

ошибка

Когда: вы можете исправить конкретный сбой и продолжить. Ловите именно этот класс, например ValueError или FileNotFoundError. Голый except: прячет и опечатки, и прерывание программы.

```
try:
    value = int(raw)
except ValueError:
    value = 0
```

*args, **kwargs и точка входа

api

Когда: вы оборачиваете чужую функцию и должны пропустить её аргументы дальше. В собственном API лучше именованные параметры: по ним видно контракт. Блок `main` не выполняется, когда файл импортируют как модуль.

```
def log(message: str, *args: object, **fields: object) → None:
    print(message, *args, fields)

def main() → None:
    log("старт", user="Борис")

if __name__ == "__main__":
    main()
```